CSE 4345 Big Data Analytics

Final exam guide · Weeks 5–10 · 13 October 2026

Chapters

WEEK 5
YARN, file formats and compression
19 pool questions · 6 topics

WEEK 6
Apache Spark: RDDs and in-memory computing
22 pool questions · 6 topics

WEEK 7
Hive, HiveQL and SQL-on-Hadoop
21 pool questions · 6 topics

WEEK 8
Data integration: Kafka, Lambda/Kappa, data lakes
23 pool questions · 6 topics

WEEK 9
Data visualization
19 pool questions · 6 topics

WEEK 10
Big data in real applications
22 pool questions · 6 topics

CHAPTER 1 · WEEK 5

YARN, file formats and compression

How a shared Hadoop cluster hands out CPU and RAM to many jobs (YARN), and how the way data is laid out and compressed on disk decides how much of it a query has to read. Part 2 tests the same ideas on LinkedIn's 5,000-node YARN cluster.

Exam question pool

19 questions from the slides · click a question for its answer

MCQ **Q1.** Which component in YARN is responsible for global resource allocation across the entire cluster?

(a) NodeManager
(b) ApplicationMaster
(c) ResourceManager
(d) NameNode

(c) ResourceManager
One RM per cluster allocates containers globally; the AM manages only its own job.

MCQ **Q2.** Which Hadoop file format stores data in columnar format and is best suited for analytical, read-heavy workloads?

(a) Avro
(b) Text/CSV
(c) Parquet
(d) JSON

(c) Parquet
Columnar, with min/max stats for predicate pushdown; Avro, CSV and JSON are row formats. ORC is also columnar: the instructor accepts it as partly correct with a justification (best for Hive), but Parquet is the broader standard.

MCQ **Q3.** What does “splittable” mean for a file format in Hadoop?

(a) The file can be deleted in parts by HDFS
(b) Multiple Mappers can process different parts of the file simultaneously
(c) The file format supports splitting schema across multiple NameNodes
(d) The file can be distributed across multiple data centres

(b) Multiple Mappers can process different parts of the file simultaneously
Reading can start anywhere in the file, which also allows data-local tasks.

MCQ **Q4.** YARN was introduced in which major Hadoop version, and what did it replace?

(a) Hadoop 1.0; replaced the NameNode
(b) Hadoop 2.0; replaced the monolithic JobTracker with a decoupled ResourceManager + ApplicationMaster
(c) Hadoop 3.0; replaced DataNodes with Containers
(d) Hadoop 2.0; replaced the Secondary NameNode with a Standby NameNode

(b) Hadoop 2.0; replaced the monolithic JobTracker with a decoupled ResourceManager + ApplicationMaster
(d) is the trap: the Standby NameNode is HDFS, not YARN.

TRUE/FALSE **1.** Parquet is a row-oriented file format optimised for write-heavy workloads.

False. Parquet is column-oriented and optimised for read-heavy analytics; the row-oriented, append-friendly format is Avro.

TRUE/FALSE **2.** YARN allows multiple computation frameworks such as Spark and Tez to share resources on the same Hadoop cluster.

True. Each framework brings its own ApplicationMaster, so YARN runs MapReduce, Spark and Tez on one cluster.

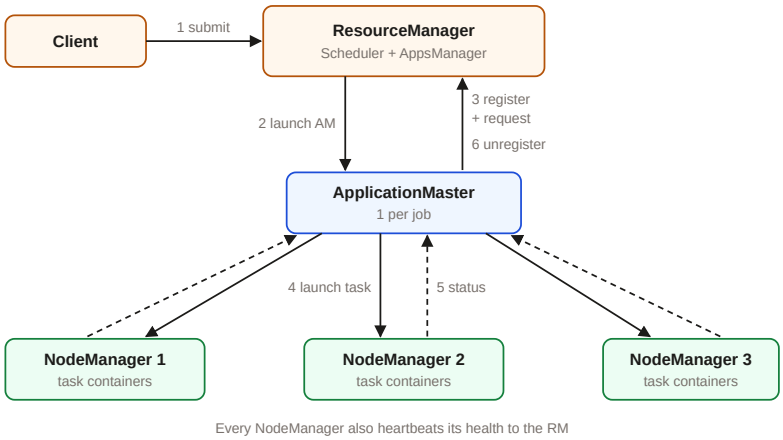
TRUE/FALSE **3.** Gzip-compressed files are natively splittable in Hadoop, allowing multiple Mappers to process them in parallel.

False. Gzip files are not splittable, so one Mapper reads the whole file; use bzip2 or Parquet + Snappy for parallel reads.

TRUE/FALSE **4.** ORC format is specifically optimised for Hive workloads and achieves among the highest compression ratios of any Hadoop format.

True. ORC is columnar, best for Hive, with the highest compression, Hive ACID and vectorized reads.

DESCRIPTIVE **D1.** Explain YARN's three-component architecture — ResourceManager, NodeManager, and ApplicationMaster. Draw a diagram showing how they interact when a Spark job is submitted to the cluster.



A job on YARN: the client submits to the ResourceManager, which starts the job's ApplicationMaster; the AM requests containers and has NodeManagers launch tasks, which report status back.

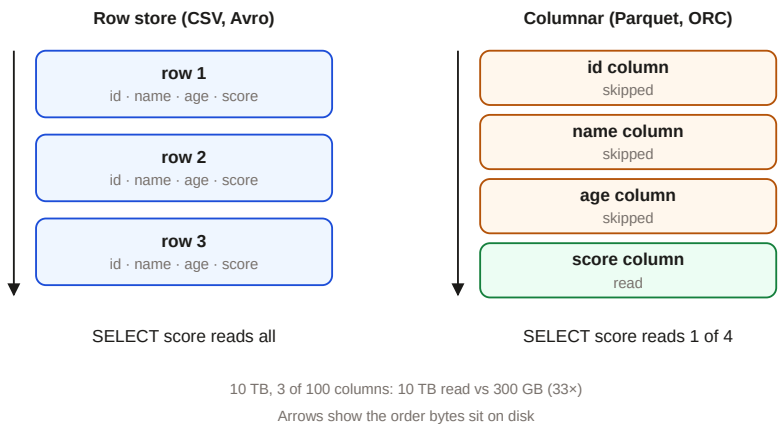
ResourceManager (1 per cluster): Scheduler grants containers; ApplicationsManager accepts jobs and starts each AM.
NodeManager (1 per node): launches and monitors containers, heartbeats to the RM, enforces CPU/RAM limits.
ApplicationMaster (1 per job): requests containers, has NMs launch tasks, retries failures, releases containers.

Diagram (see the diagram in the YARN and its three components card): Client, RM, Spark AM, NM1–NM3, with numbered arrows:

1. Client submits the job to the RM.
2. RM launches the Spark AM in a container.
3. AM registers with the RM and requests containers; the RM Scheduler grants them.
4. AM tells NM1–NM3 to launch executors.
5. Executors report status to the AM.
6. AM unregisters from the RM when done.

Example: the Spark AM asks for 100 containers {4 cores, 8 GB}; the busy RM grants 80 and the AM runs executors in them.

DESCRIPTIVE **D2.** What is columnar storage? Why does it improve performance for analytics queries that select only a subset of columns? Use a concrete example with numbers to illustrate the I/O reduction.



A row store keeps whole records together, so reading one column reads them all; a columnar store keeps each column together and skips the rest.

Columnar storage keeps all values of one column together on disk (Parquet, ORC); a row store (CSV, Avro) writes whole records one after another.

Why faster: the query reads only the columns it names; Parquet's min/max stats skip whole row groups (predicate pushdown); same-type values compress better.

Example: 100 columns, 10 TB, query on 3 columns.

Format	Data read
CSV	10 TB
Parquet	10 × 3/100 = 0.3 TB = 300 GB

I/O cut 97%, about **33×** faster.

DESCRIPTIVE **D3.** Compare Avro and Parquet. Explain what “schema evolution” means and why Avro handles it better. In which scenarios would you use each?

	Avro	Parquet
Layout	row	columnar
Strength	schema evolution, appends	analytic reads, pushdown
Codec	Snappy/Deflate	Snappy/Gzip

Schema evolution: the schema changes over time (e.g. a new field) while old files and readers keep working.

Why Avro handles it: the writer schema is stored with the data, the reader supplies its own schema, fields match by name; missing fields get defaults, unknown fields are ignored. Backward and forward compatible.

Use: Avro for Kafka → HDFS ingest and data exchange; Parquet for Spark, Presto and ML analytics.

ANALYTICAL **A1.** Data Lake Performance Migration: A Dhaka-based telecom company stores 10 TB of daily call-detail records (CDR) on HDFS as gzip-compressed CSV files. Their data engineers report that a Hive query selecting 4 of 80 columns takes 6 hours. – Identify two specific reasons why this configuration is slow – Propose a migration strategy: new format, compression codec, and partitioning scheme – Estimate the expected I/O reduction (show your calculation) – What YARN scheduler would you use and why, given that multiple teams share the cluster?

i) Why slow

1. Gzip is not splittable: one Mapper per file, the cluster idles.
2. CSV is a row store: reading 4 columns parses all 80; no stats to skip data.

ii) Migration: ORC (Hive) or Parquet (if Spark reads it too); Snappy for hot data (Zlib for ORC in Hive, Zstd for archive); partition by date and region, e.g. `dt=YYYY-MM-DD/region=` ; convert once, then nightly.

iii) I/O estimate

Fraction read	4/80 = 0.05
Data read	10 TB × 0.05 = 500 GB
Reduction	**95%** (20× less)
Time if I/O-bound	360 min × 0.05 ≈ **18 min**

iv) Scheduler: Capacity: guaranteed queues per team (ETL, analysts, data science), idle capacity borrowed, so no team starves another. FIFO lets one long job block all.

ANALYTICAL **A2.** YARN Failure Trace: A NodeManager (NM2) hosting 5 running containers for a MapReduce job fails mid-execution. Trace the complete failure recovery sequence: which components detect the failure, in what order, what state changes occur in the ResourceManager, and how are the 5 containers’ tasks restarted.

1. NM2 crashes; its 5 containers die; heartbeats stop.

2. **RM** detects the missed heartbeats after the timeout (>10 min) and declares NM2 lost.
3. **RM state:** NM2 removed from schedulable capacity; its 5 containers marked **failed**.
4. RM notifies the MapReduce **AM**.
5. AM requests 5 replacement containers.
6. RM grants them on healthy nodes (e.g. NM1, NM3), preferring nodes with HDFS replicas of the input.
7. AM has those NMs relaunch the 5 tasks from their input splits; finished maps whose output was on NM2's disk may rerun.

If the AM was on NM2, the RM restarts it (default 2 attempts); MapReduce recovers state from HDFS checkpoints.

DISCUSSION **L1.** Resource consolidation increases utilisation: merging 3 siloed clusters into one raised average utilisation from 45% to 78% — the same capacity served 73% more workload

Claim: one shared cluster uses hardware better than three silos.

For: LinkedIn's silos (MapReduce, Spark, Samza) were idle at different times; merged, utilisation rose 45% → 78%, and 78/45 = 1.73, so 73% more work on the same hardware. No cross-cluster copying, one config instead of three.

Against: workloads now compete, so queues, pre-emption and RM high availability become essential.

DISCUSSION **L2.** Capacity Scheduler enables SLAs: production queues guaranteed minimum resources; analyst adhoc queries cannot starve revenue-generating pipelines

Claim: guaranteed queues protect SLAs (promised finish times).

For: production holds 40% at LinkedIn, so the 10% adhoc queue cannot delay revenue pipelines, while idle capacity is still lent out.

Against: fixed percentages need tuning, and analysts may wait at busy times; Fair gives short jobs faster turnaround.

DISCUSSION **L3.** Pre-emption needs tuning: aggressive pre-emption killed long-running Spark jobs; LinkedIn set 30-second grace periods and added Spark checkpoint-on-preempt hooks

Claim: pre-emption is needed but must match the workload.

For: without reclaiming borrowed containers, queue guarantees are empty promises.

Against: killing a container wastes its work; long Spark jobs restarted repeatedly. LinkedIn's fix: a 30-second grace period and checkpoint-on-preempt hooks so jobs resume.

DISCUSSION **L4.** YARN is the OS; frameworks are processes: the paradigm shift — the cluster is the computer, YARN is its kernel

Claim: YARN is to a cluster what an OS kernel is to a computer.

For: the RM allocates CPU and RAM as containers, NMs isolate them with cgroups, and frameworks are just applications with their own AM; the same three-tier design lives on in Kubernetes.

Against: YARN sizes containers only by CPU and RAM, relies on HDFS for storage, and Kubernetes has replaced it for most new deployments.

DISCUSSION **L5.** Samza on YARN: stream processing is just another set of long-running containers; YARN's fault tolerance restarts failed Samza tasks

Claim: a stream job is just long-running containers.

For: Samza shares the cluster in its 15% queue, no separate stream cluster, and YARN restarts failed tasks on other nodes.

Against: the containers hold capacity permanently, and the >10 min NodeManager timeout is slow for real-time data, so Samza must resume from the right Kafka offset itself.

LIVE EXERCISE

LE. Task (20 minutes, pairs) Model LinkedIn's 5,000-node YARN cluster in Python. Given: • 5,000 nodes; each: 64 GB RAM, 16 vCPUs • Capacity Scheduler: production 40%, spark 25%, samza 15%, adhoc 10%, reserve 10% • Container sizes: Hive mapper: {4 GB, 2 vCPU}; Spark executor: {8 GB, 4 vCPU}; Samza task: {2 GB, 1 vCPU} Questions: 1 Max concurrent containers per queue? 2 Which resource is the bottleneck (CPU or RAM)? 3 If production needs 10,000 Hive mappers, how many nodes are required?

```
NODES    = 5_000
RAM_GB   = 64
VCPUS    = 16

queues = {
  "production": 0.40,
  "spark":      0.25,
  "samza":      0.15,
  "adhoc":      0.10,
  "reserve":    0.10,
}
containers = {
  "production": {"ram": 4, "cpu": 2},
  "spark":      {"ram": 8, "cpu": 4},
  "samza":      {"ram": 2, "cpu": 1},
  "adhoc":      {"ram": 4, "cpu": 2},
}

for q, frac in queues.items():
    nodes_q = NODES * frac
    ram_q    = nodes_q * RAM_GB
    cpu_q    = nodes_q * VCPUS
    if q in containers:
        c = containers[q]
        by_ram = ram_q // c["ram"]
        by_cpu = cpu_q // c["cpu"]
        max_c = int(min(by_ram, by_cpu))
        bottleneck = ("RAM" if by_ram < by_cpu
                            else "CPU")
        print(f"{q}: {max_c} containers "
              f"(bottleneck: {bottleneck})")
```

Task: run the skeleton; add Q3:

```
import math
c = containers["production"]
per_node = min(RAM_GB // c["ram"], VCPUS // c["cpu"])
print(per_node, math.ceil(10_000 / per_node))
```

Expected output:

```
production: 16000 containers (bottleneck: CPU)
spark: 5000 containers (bottleneck: CPU)
samza: 12000 containers (bottleneck: CPU)
adhoc: 4000 containers (bottleneck: CPU)
8 1250
```

1. production 16,000; spark 5,000; samza 12,000; adhoc 4,000; reserve has no container size, so none.

2. CPU. A node has 64/16 = 4 GB per vCPU; every container asks 2 GB per vCPU, so vCPUs run out with half the RAM idle.

3. Per node min(64/4, 16/2) = 8 mappers; 10,000 / 8 = **1,250 nodes**, which fits in production's 2,000.

CHAPTER 2 · WEEK 6

Apache Spark: RDDs and in-memory computing

Spark keeps working data in RAM so iterative jobs stop paying MapReduce's disk cost on every pass. The chapter covers how Spark is built, how RDDs recover from failures through lineage, lazy evaluation, the narrow/wide split that decides shuffles and stages, and DataFrames.

Exam question pool

22 questions from the slides · click a question for its answer

MCQ **Q1.** What makes Spark significantly faster than MapReduce for iterative algorithms?

(a) Spark uses more CPU cores per node
(b) Spark writes intermediate results to faster SSDs
(c) Spark caches intermediate data as RDDs in memory
(d) Spark skips the shuffle phase entirely

(c) Spark caches intermediate data as RDDs in memory

Later iterations read from RAM instead of re-reading and re-writing HDFS; the hardware is the same.

MCQ **Q2.** How does Spark achieve fault tolerance in RDDs without full data replication?

(a) Checkpointing to HDFS every 60 seconds
(b) Tracking lineage and recomputing lost partitions from parent RDDs
(c) Copying each RDD partition twice across two Executors
(d) Using a write-ahead log on the Driver node

(b) Tracking lineage and recomputing lost partitions from parent RDDs

The recipe is kept, not copies; (c) is replication.

MCQ **Q3.** Which of the following is **NOT** a component of the Spark ecosystem?

(a) MLlib
(b) GraphX
(c) Structured Streaming
(d) HBase

(d) HBase

HBase is a separate NoSQL database on HDFS; Spark can only read it as a data source.

MCQ **Q4.** In Spark, a “transformation” such as map() or filter() is:

(a) Immediately executed when called and materialised to memory
(b) Lazily evaluated — only executed when a subsequent action is called
(c) Automatically persisted to HDFS after execution
(d) Only available in the Scala API, not in PySpark

(b) Lazily evaluated — only executed when a subsequent action is called

`rdd.filter(f)` alone does nothing; `rdd.filter(f).count()` runs it.

TRUE/FALSE **1.** Spark can only run on top of Hadoop YARN and has no other deployment option.

False. The Cluster Manager can be YARN, Kubernetes, Mesos or Spark Standalone.

TRUE/FALSE **2.** RDDs are mutable — individual elements can be modified in place after creation.

False. RDDs are immutable; every transformation creates a new RDD.

TRUE/FALSE **3.** Spark's DataFrame API provides schema-aware, named-column operations similar to SQL tables.

True. A DataFrame has named columns and a schema, like a relational table, and accepts SQL.

TRUE/FALSE **4.** MapReduce is generally better than Spark for machine learning workloads that require many iterations over the same dataset.

False. Spark is better: it caches the data in RAM and is 10–100× faster on iterative work, while MapReduce hits disk every iteration.

DESCRIPTIVE **D1.** Explain the concept of a **Resilient Distributed Dataset (RDD)**. What does *"resilient"* mean in this context? How does Spark recover a lost RDD partition without replication?

RDD: a read-only, partitioned collection of records, created from stable storage (HDFS, S3) or by transforming another RDD. Properties: immutable, partitioned, lazy, lineage, resilient.

Resilient: it survives a node failure by recomputing lost partitions from its lineage (the steps that built it) instead of keeping 3 copies. The computation is the backup.

Recovery:

1. Spark detects Part 2 of mappedRDD is lost.
2. Traces lineage: mappedRDD ← filteredRDD ← inputRDD.
3. Re-reads input Part 2 from HDFS.
4. Re-applies filter, then map.
5. Part 2 restored; the job is not restarted.

Diagram: draw the lineage chain shown in the diagram in the RDD card: inputRDD → filteredRDD → mappedRDD, with Part 2 of mappedRDD crossed out and the recovery path going back to inputRDD Part 2 on HDFS.

Cheap for narrow dependencies; checkpoint long chains.

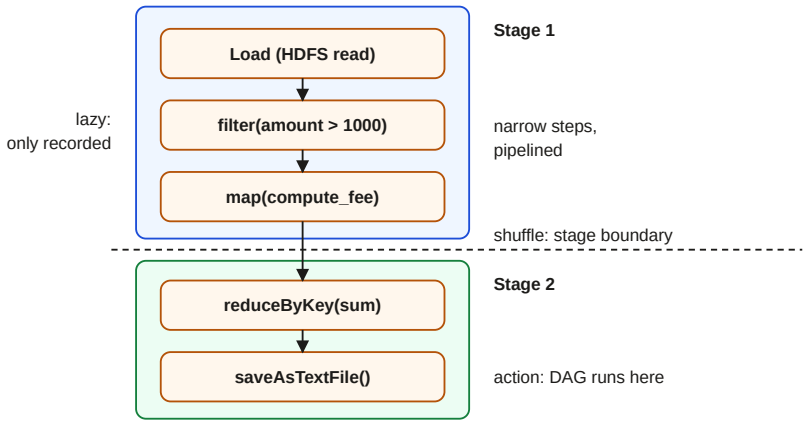
DESCRIPTIVE **D2.** Differentiate between **Spark transformations** and **actions**. Give two concrete examples of each. Why does Spark not execute transformations immediately?

	Transformation	Action
Runs	Lazily, only recorded in the DAG	Immediately, triggers the DAG
Returns	A new RDD	A value to the Driver, or output to storage
Examples	`map(f)`, `filter(f)`	`count()`, `collect()`

Why wait: with the whole DAG in view, Spark can

- optimise the plan (DAG scheduler, or Catalyst for DataFrames);
- pipeline filter and map in one pass, with no intermediate disk writes;
- run only the partitions needed, e.g. `first()` stops at the first match.

DESCRIPTIVE **D3.** Describe Spark's **lazy evaluation model**. What is the advantage of deferring computation until an action is called? Use the example `rdd.filter(x > 1000).first()` to illustrate.



Transformations are only recorded; the action saveAsTextFile runs the DAG, with narrow steps pipelined in Stage 1 and the reduceByKey shuffle starting Stage 2.

Model: transformations are only recorded in the DAG. When an action is called, Spark analyses the DAG, optimises it, pipelines compatible steps in one pass per partition, and runs only the partitions it needs.

Advantages: less work (early stop, skipped partitions), no intermediate disk writes, whole-plan optimisation.

Example:

1. `filter(x > 1000)` : recorded, no data read.
2. `first()` : action; Spark starts reading the first partition.
3. It returns the first value above 1000 and stops.
4. Eager execution would filter the entire dataset, then keep one value.

ANALYTICAL

A1. Iterative I/O Cost Analysis: You need to train a K-Means clustering model over **50 GB** of bKash transaction data with **100 iterations**. Assume HDFS read speed = 500 MB/s and RAM cache read speed = 5,000 MB/s.

- Calculate the total HDFS I/O for MapReduce (all 100 iterations)
- Calculate the total I/O for Spark (50 GB fits in cluster RAM)
- Estimate the wall-clock time for each approach
- Under what conditions would Spark NOT outperform MapReduce?

Given: D = 50 GB = 50,000 MB, k = 100, HDFS 500 MB/s (write assumed = read), RAM 5,000 MB/s.

	MapReduce	Spark
HDFS I/O	100 × (50 + 50) = **10 TB**	**50 GB** (read once); RAM reads 99 × 50 = 4,950 GB
Per pass	50,000/500 = 100 s read + 100 s write = 200 s	100 s first, then 50,000/5,000 = 10 s
Total time	100 × 200 = **20,000 s ≈ 5.56 h**	100 + 99 × 10 = **1,090 s ≈ 18.2 min**

Disk I/O 200× less; time speedup 20,000 / 1,090 ≈ **18.3×**.

Spark does not win when: the job is single-pass (nothing to reuse); the data does not fit in RAM (spill or evict and recompute); Executor memory (~8 GB minimum) is too costly; stable MapReduce code is already fast enough.

ANALYTICAL

A2. RDD Failure Recovery Trace: A Spark job computes `input` → `filteredRDD` → `mappedRDD` → `reducedRDD` over 10 partitions. Mid-execution, the Executor holding partitions 3 and 7 of `reducedRDD` crashes. Trace the complete recovery sequence: what does the Driver detect, which partitions of which RDDs need to be recomputed, and what is the worst-case recovery cost if `reducedRDD` involves a **wide** dependency (`groupByKey`)?

Stages: filter, map narrow → Stage 1 (input → filteredRDD → mappedRDD). groupByKey wide → Stage 2 (reducedRDD).

Driver detects: the Executor is lost; tasks for reducedRDD partitions 3 and 7 failed. The other 8 partitions are fine.

Lineage: reducedRDD ← groupByKey ← mappedRDD ← map ← filteredRDD ← filter ← input.

Case	Recomputed
If all narrow	Partitions 3 and 7 at each level: 2 × 4 = 8 computations (20%)
Wide, shuffle files survive	Only reducedRDD 3 and 7, re-fetching from all 10 mappedRDD outputs
Wide, worst case	Shuffle output lost too: all 10 input, filteredRDD and mappedRDD partitions (30) + reshuffle + reducedRDD 3, 7 (2) ≈ the whole job

Limit the cost: reduceByKey instead of groupByKey, cache mappedRDD, checkpoint long chains.

DISCUSSION

1. In-memory is a multiplier, not a panacea: Spark is 10–100× faster on iterative workloads; for single-pass ETL over data larger than RAM, gains are smaller (2–5×)

Claim: memory speeds up reuse; it does not fix every workload.

For: iterative jobs reuse the working set. RDD paper: logistic regression 127 s vs 0.96 s per iteration (132×).

Against: single-pass ETL reads each byte once, and data larger than RAM spills to disk, so the gain falls to 2–5×.

Choose by workload: more than 2 iterations favours Spark.

DISCUSSION **2. The lineage model scales elegantly**: re-computing a lost partition from lineage is cheaper than maintaining 3× replicated in-memory state

Claim: lineage beats replication for in-memory data.

For: 3× replication triples RAM and network writes on every step; lineage costs nothing until a failure and then rebuilds only lost partitions.

Against: with wide dependencies or long chains, one lost partition can need many ancestors recomputed, so checkpoints are still needed.

DISCUSSION **3. Shuffle is the bottleneck**: both at GraySort and in production; minimising wide dependencies (shuffle) is the primary Spark tuning skill

Claim: shuffles dominate Spark's cost.

For: a shuffle moves data over the network, writes to disk and forces a stage barrier. Spark's GraySort win came from a cheaper shuffle: sort-based shuffle (about 120 TB less I/O), all 8 SSDs as shuffle directories, one sortByKey. Tuning: reduceByKey over groupByKey, co-partition before joins.

Against: sorts and un-partitioned joins need a shuffle; you minimise it, you cannot remove it.

DISCUSSION **4. Benchmark marketing is real**: GraySort made Spark famous; it also hid cases where Spark's JVM memory overhead hurt performance (many small files, high-cardinality `groupBy`)

Claim: a headline benchmark sells a system but shows one workload.

For: 100 TB in 23 min on 206 nodes vs Hadoop's 72 min on 2,100 drove Spark's adoption.

Against: it ran on tuned large nodes (244 GB RAM, 8 SSDs). Many small files (per-task overhead) and high-cardinality groupBy (too many keys for JVM memory, GC pauses) were not tested.

Test on your own workload.

DISCUSSION **5. Open-source velocity**: the Apache Spark community grew to 1,000+ contributors in 5 years; Databricks' business model (cloud + support) aligned commercial incentives with open-source development

Claim: a free engine plus a paid service sped up Spark.

For: Databricks earns from cloud and support, not licences, so improving open Spark helps its business. 1,000+ contributors in 5 years produced Spark SQL, MLlib, Structured Streaming, GraphX.

Against: one company steering the project can shape the roadmap and push features to its paid product first (lock-in risk).

LIVE EXERCISE

Task. Task (25 minutes, pairs)
Run the following PySpark script locally (Spark local mode).
Goal: build and inspect an RDD lineage; observe lazy evaluation; compare cached vs. uncached timing.

```
from pyspark import SparkContext
import time
sc = SparkContext("local[4]", "RDDDemo")
sc.setLogLevel("WARN")
# Build lineage (lazy - no compute yet)
lines = sc.textFile("README.md")
words = lines.flatMap(lambda l: l.split())
pairs = words.map(lambda w: (w.lower(), 1))
counts = pairs.reduceByKey(lambda a, b: a+b)
# Inspect lineage BEFORE action
print(counts.toDebugString().decode())
# First action - triggers full DAG
t0 = time.time()
n = counts.count()
print(f"count={n}  time={time.time()-t0:.2f}s")
# Cache and re-run
counts.cache()
counts.count()          # warm up cache
t1 = time.time()
n = counts.count()      # served from cache
print(f"cached count={n} "
      f"time={time.time()-t1:.4f}s")
sc.stop()
```

`local[4]` = local mode, 4 threads. textFile, flatMap, map, reduceByKey only build lineage; the first `count()` runs the DAG. Key additions for the questions (run with `python rdd_demo.py`):

```
print(counts.toDebugString().decode())   # Q1 before cache
for i in (1, 2):                          # Q2 two uncached runs
    t0 = time.time(); counts.count()
    print(i, time.time() - t0)
counts.cache()
print(counts.toDebugString().decode())   # Q1 after cache
counts.count()                            # fills the cache
t1 = time.time(); counts.count()
print("cached", time.time() - t1)
# Q3: counts = pairs.groupByKey().mapValues(sum)
input("open localhost:4040, then Enter")
```

Expected: the lineage shows HadoopRDD → Python RDDs, with a `+-` indent at the reduceByKey shuffle; count = number of distinct lower-cased words.

LIVE EXERCISE

1. Call `rdd.toDebugString()` before and after `cache()` — what changes?

The parent chain does not change, because cache() is lazy and lineage is kept for recovery. After cache() the top line shows a storage level (e.g. `[Memory Serialized 1x Replicated]`). After an action fills the cache it also shows `CachedPartitions` and memory size (expected).

LIVE EXERCISE

2. Time `rdd.count()` twice without `cache()`. Then add `.cache()` and time again. Report the speedup.

Speedup = uncached time / cached time. Expected: run 1 slowest (start-up, read, shuffle); run 2 a bit faster (shuffle files reused, Stage 1 skipped); cached run fastest. On a small README start-up overhead dominates, so the ratio is modest; the RDD paper shows 127 s → 0.96 s (132×) at scale.

LIVE EXERCISE

3. Change `reduceByKey` to `groupByKey`. Use the Spark UI (localhost:4040) to compare shuffle read/write bytes between the two.

Use `pairs.groupByKey().mapValues(sum)`, then read Shuffle Write/Read on the UI's Stages tab. Expected: same counts, larger shuffle bytes for groupByKey, because reduceByKey combines each partition's pairs before the shuffle and groupByKey sends every `(word, 1)` pair.

CHAPTER 3 · WEEK 7

Hive, HiveQL and SQL-on-Hadoop

Hive lets analysts query data on HDFS with SQL-like HiveQL, which it compiles into MapReduce, Tez or Spark jobs, using a Metastore that records every table's schema and location. The chapter covers how Hive runs a query, schema-on-read, partitioning and bucketing for speed, Presto for interactive queries, and Spotify's Discover Weekly pipeline as a case study.

Exam question pool

21 questions from the slides · click a question for its answer

MCQ **Q1.** In Hive, where is table schema metadata (column names, types, partition info) stored?

(a) HDFS NameNode
(b) Hive Metastore (relational DB such as MySQL)
(c) HBase
(d) ZooKeeper

(b) Hive Metastore (relational DB such as MySQL)
The NameNode holds HDFS block metadata, not table schemas.

MCQ **Q2.** What does "schema-on-read" mean in the context of Hive?

(a) Schema is validated when data is inserted into the table
(b) Schema is applied when data is queried, not when it is stored on HDFS
(c) Schema is stored separately from data in HDFS metadata
(d) Schema is inferred automatically from column names in the file header

(b) Schema is applied when data is queried, not when it is stored on HDFS
(a) describes schema-on-write, how MySQL works.

MCQ **Q3.** Which Hive optimization technique physically organises data into separate HDFS sub-directories by a column's value to avoid full table scans?

(a) Bucketing
(b) Indexing
(c) Partitioning
(d) Compression

(c) Partitioning
Bucketing splits files inside a partition; it does not skip directories.

MCQ **Q4.** Presto's primary performance advantage over Hive+MapReduce for interactive queries is:

(a) Presto supports more SQL functions than HiveQL
(b) Presto has better native integration with HBase and Cassandra
(c) Presto executes queries entirely in memory without writing intermediate results to disk
(d) Presto has native Python UDF support

(c) Presto executes queries entirely in memory without writing intermediate results to disk
Hive+MR writes to HDFS between every stage.

TRUE/FALSE **1.** HiveQL supports full ACID transactions on all table types, just like MySQL.

False. Corrected: HiveQL supports ACID only on ACID-enabled transactional ORC tables; MySQL (InnoDB engine) gives full ACID: INSERT, UPDATE, DELETE and ROLLBACK on any row.

TRUE/FALSE **2.** Partitioning in Hive reduces query time by eliminating irrelevant data partitions from the HDFS scan.

True. A WHERE filter on the partition key makes Hive read only the matching sub-directories (partition pruning).

TRUE/FALSE **3.** Hive directly executes a HiveQL query on HDFS without any intermediate translation step.

False. Corrected: Hive compiles HiveQL (parse, analyse, plan, optimise) into MapReduce, Tez or Spark jobs, which then read HDFS.

TRUE/FALSE **4.** Bucketing in Hive is particularly useful for optimising JOIN operations between two large tables.

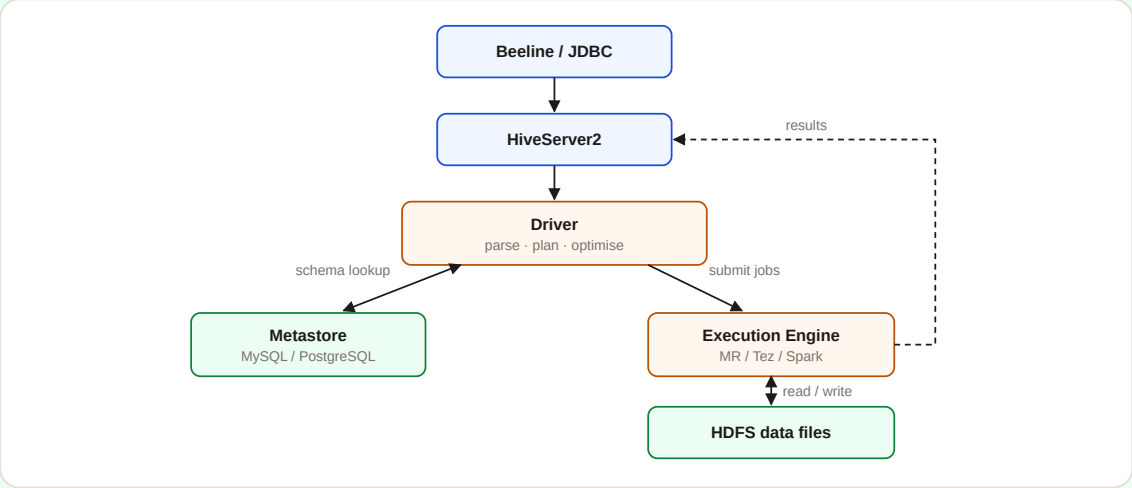
True. Both tables bucketed on the join key with the same N let Hive join bucket i with bucket i, with no full shuffle.

DESCRIPTIVE **D1.** Explain the Hive architecture: what is the role of HiveServer2, the Driver, and the Metastore? How is a HiveQL query compiled into executable jobs? Draw the component diagram.

- **HiveServer2**: JDBC/ODBC front door; authenticates users, accepts HiveQL from Beeline, BI tools, JDBC.
- **Driver**: Parser → Semantic Analyser → Logical Planner → Optimizer → Physical Plan Generator; manages the query lifecycle.
- **Metastore**: MySQL/PostgreSQL catalogue of schemas, types, partitions, HDFS paths.

Compilation: parse to AST → check names and types in the Metastore → logical plan (Filter, Join, GroupBy, OrderBy) → optimise (partition pruning) → MR/Tez/Spark jobs → submit to YARN; results to HDFS and client.

Diagram:



DESCRIPTIVE **D2.** Differentiate between partitioning and bucketing in Hive. When would you choose each? Describe a scenario in Bangladesh e-commerce data where both are used together.

	Partitioning	Bucketing
Unit	Directory per value	Fixed N files, hash(col) % N
Benefit	Pruning for WHERE filters	Bucket map join
Column	Low cardinality (date, division)	High cardinality key (customer_id)

Choose partitioning when queries filter on a few-valued column; bucketing when large tables are joined on a many-valued key.

Scenario: a Bangladeshi e-commerce `orders` table, `PARTITIONED BY (order_date, division)` and `CLUSTERED BY (customer_id) INTO 32 BUCKETS`. The Dhaka daily report reads one folder; the join with `customers` (also 32 buckets on customer_id) pairs bucket i with i.

DESCRIPTIVE **D3.** What is "schema-on-read"? What are its advantages and disadvantages compared to "schema-on-write" in a traditional RDBMS such as MySQL? Give one example where schema-on-read causes a data quality problem.

Schema-on-read: Hive applies column types at query time; loading is a raw file copy. MySQL (schema-on-write) checks each row at INSERT.

Advantages	Disadvantages
Fast load, no validation	Slower queries (parse every time)
Load before the schema is known; many schemas on the same files; no ETL at ingest	Bad data silent: becomes NULL
Open formats (ORC, Parquet, CSV, JSON, Avro)	Unfit for OLTP (banking, orders)

Example: `abc` in a DOUBLE `amount` column. MySQL rejects the INSERT; Hive stores it and returns NULL, so `SUM(amount)` under-reports revenue with no warning. Fix: Great Expectations or dbt tests downstream.

ANALYTICAL

A1. Partitioning & Bucketing Design: You have a 5 TB e-commerce transaction table with columns: (txn_id, user_id, product_id, amount, country, txn_date). Design a complete partitioning and bucketing strategy to optimise: – Daily revenue reports filtered by country and txn_date – JOINs between this table and a 10 M-row user profile table (keyed on user_id). Write the complete CREATE TABLE DDL. Justify why you chose a specific number of buckets and which column(s) you partitioned on.

Partition on `txn_date, country` (low cardinality, used in report filters). **Bucket** on `user_id` (high-cardinality join key), same N on both tables for a bucket map join. Never partition on user_id or txn_id.

Bucket count (assume 3 years, 5 countries):

Partitions	3 × 365 × 5 = 5,475
Per partition	5,000 GB / 5,475 ≈ 0.913 GB
8 buckets	≈ 114 MB/file (near 128 MB block), 43,800 files
32 buckets	≈ 28.5 MB/file, 175,200 files: too small

```
CREATE TABLE transactions (
  txn_id BIGINT, user_id BIGINT,
  product_id BIGINT, amount DOUBLE)
PARTITIONED BY (txn_date STRING, country STRING)
CLUSTERED BY (user_id) SORTED BY (user_id) INTO 8 BUCKETS
STORED AS ORC;

CREATE TABLE user_profile (
  user_id BIGINT, name STRING, city STRING)
CLUSTERED BY (user_id) SORTED BY (user_id) INTO 8 BUCKETS
STORED AS ORC;
```

Result: a one-day, one-country report reads 1 of 5,475 partitions (≈ 0.018%); the join pairs bucket i with i.

ANALYTICAL

A2. Query Compilation to MapReduce: Write HiveQL to find the top 5 products by total revenue in 2023. Then trace how Hive compiles this query into MapReduce stages: what is emitted by the Map phase, what is the shuffle key, and what does the Reduce phase compute? How would switching the execution engine from MapReduce to Tez change the execution plan?

```
SELECT product_id, SUM(amount) AS total_revenue
FROM transactions
WHERE YEAR(txn_date) = 2023
GROUP BY product_id
ORDER BY total_revenue DESC
LIMIT 5;
```

	Job 1: aggregate	**Job 2: sort + limit**
Map emits	(product_id, amount) for 2023 rows; combiner gives partial sums	(total_revenue, product_id); each mapper keeps its top 5
Shuffle key	product_id	total_revenue DESC, to one reducer
Reduce	SUM(amount) per product, written to HDFS	First 5 rows

Tiny check: (P1,100), (P2,50), (P1,30), (P3,80), (P2,20) → P1 130, P3 80, P2 70.

Tez: one DAG, Map 1 → Reducer 2 → Reducer 3. No HDFS write between the jobs, one submission, container reuse. Same result, faster.

DISCUSSION **1.** Spotify's co-occurrence query runs GROUP BY song_a, song_b over 2 billion playlist rows. Estimate the output size if there are 30M songs. Propose a partitioning + bucketing scheme for the playlist_songs Hive table that would reduce the shuffle size for this query. [CLO4 – Apply]

Claim: output is far below the 30M-song bound. Bound: 30M × (30M − 1) / 2 ≈ 4.5 × 10^14 pairs. Realistic (assume 20 songs per playlist): 10^8 playlists × 190 = 1.9 × 10^10 pairs, ≤ 304 GB at 16 B/row, much less after HAVING > 5.

Scheme: `PARTITIONED BY (week)`, `CLUSTERED BY (playlist_id) SORTED BY (playlist_id, song_id) INTO 1024 BUCKETS`. **For:** the self-join runs inside each bucket with no shuffle, and weekly partitions cut 8 h to 2 h. **Against:** it does not shrink the GROUP BY shuffle on song pairs; rely on map-side aggregation there.

DISCUSSION **2.** The ALS collaborative filtering model is a matrix factorisation over a 75M-user × 30M-song matrix. At 4 bytes per float and rank k = 50, calculate the model size and explain whether it fits in a 5,000-node YARN cluster's aggregate RAM. [CLO4 – Analyse]

Size: (75M + 30M) × 50 × 4 B = **21 GB** (users 15 GB + songs 6 GB).

Fits? Assume 64 GB/node: 5,000 × 64 GB = 320 TB; 21 GB ≈ 0.0066%, about 4.2 MB per node. **For:** it fits easily, even on one large server; the dense 9 PB matrix would not. **Against:** training also holds the interaction data, shuffled factor blocks and JVM overhead, several times the model size, but still well within the cluster.

DISCUSSION **3.** Spotify uses Luigi while other companies use Apache Airflow. Compare the two on: dependency declaration style, backfill capability, and UI/observability. Which would you choose for a 100-step daily pipeline? [CLO3 – Evaluate]

Claim: choose Airflow for a 100-step daily pipeline. Luigi: target-based `requires()/output()`, skips finished outputs, backfill by date parameters, simple UI. Airflow: DAG with a scheduler, built-in catchup/backfill, rich UI with logs, retries and alerts.

For: at 100 steps you need to see which step failed, read its logs and re-run past days, which Airflow gives out of the box. **Against:** Luigi is lighter and its skip-if-output-exists model fits file-based Hadoop pipelines like Spotify's.

LIVE EXERCISE

Task. Task (25 minutes, pairs): Simulate a Hive session using PySpark's SQL interface (which reads the same Hive Metastore). Dataset: a 10,000-row synthetic play_events CSV (generate with the skeleton below).

```
from pyspark.sql import SparkSession
import random, datetime

spark = SparkSession.builder \
        .appName("HiveDemo") \
        .getOrCreate()
spark.setLogLevel("WARN")

# Generate synthetic data
rows = []
for i in range(10_000):
    rows.append({
      "user_id":    random.randint(1, 500),
      "song_id":    random.randint(1, 200),
      "playlist_id":random.randint(1, 1000),
      "dt":         "2024-10-01"
    })

df = spark.createDataFrame(rows)
df.createOrReplaceTempView("play_events")

# Co-occurrence query
result = spark.sql("""
    SELECT a.song_id AS song_a,
           b.song_id AS song_b,
           COUNT(*) AS co_count
    FROM play_events a
    JOIN play_events b
      ON a.playlist_id = b.playlist_id
     AND a.song_id < b.song_id
    GROUP BY a.song_id, b.song_id
    HAVING COUNT(*) > 2
    ORDER BY co_count DESC
""")
result.show(10)
```

Write the data as a real partitioned table, then time and inspect the queries.

```
import time
random.seed(42)  # rows as in the skeleton, dt from 2 dates
df.write.mode("overwrite").partitionBy("dt") \
  .format("parquet").saveAsTable("play_events_part")
t0 = time.time()
spark.sql(cooc_sql).show(10)      # skeleton query on play_events_part, with a.dt = b.dt = '2024-10-01'
print(time.time() - t0)
songs = spark.createDataFrame(
  [(i, "genre" + str(i % 10)) for i in range(1, 201)], ["song_id", "genre"])
songs.createOrReplaceTempView("songs")   # 200-row dimension table
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)
hint_sql = """SELECT /*+ BROADCAST(s) */ s.genre, COUNT(*)
  FROM play_events_part p JOIN songs s ON p.song_id = s.song_id
  WHERE p.dt = '2024-10-01' GROUP BY s.genre"""
spark.sql(hint_sql).explain(mode="formatted")
```

Expected with the single-date skeleton: about 10 rows per playlist, about 50,000 pair rows, about 9,000 pairs kept by HAVING > 2, top count about 11 to 14 (simulated; varies with the data).

LIVE EXERCISE

1. Create a partitioned Spark table (partition on dt).

`df.write.partitionBy("dt").saveAsTable("play_events_part")` writes one sub-directory per date, e.g. `play_events_part/dt=2024-10-01/`, like Hive's `year=2024/`. Expected `SHOW PARTITIONS`: one row per date.

LIVE EXERCISE **2.** Run the co-occurrence query and time it.

Wrap an action (`show` , `count`) in `time.time()` : Spark is lazy, so `spark.sql()` alone runs nothing. Expected: a few seconds on a laptop, mostly start-up and the join/GROUP BY shuffle. The self-join grows with the square of playlist length (10 songs give 45 pairs).

LIVE EXERCISE **3.** Add a broadcast join hint for the songs dimension table and compare timings.

`/*+ BROADCAST(s) */` copies the 200-row songs table to every executor, so the big table is not shuffled (Hive's map-side join). The plan changes from SortMergeJoin to BroadcastHashJoin. Expected: slightly faster on 10,000 rows. Spark broadcasts tables under 10 MB automatically, so set the threshold to -1 to compare.

LIVE EXERCISE **4.** Use EXPLAIN to print the physical plan and identify where partition pruning occurs.

Run `EXPLAIN FORMATTED` or `df.explain(mode="formatted")` . Pruning shows in the FileScan node as `PartitionFilters: [isnotnull(dt), (dt = 2024-10-01)]` : only that folder is read. Filters on ordinary columns appear as `PushedFilters` . A temp view built in memory shows no PartitionFilters.

CHAPTER 4 · WEEK 8

Data integration: Kafka, Lambda/Kappa, data lakes

How data moves from the systems that create it to the systems that analyse it: batch tools (Sqoop, Flume) and the Kafka event log. How Lambda and Kappa architectures combine batch and stream processing, and how a data lake stays usable instead of turning into a swamp.

Exam question pool

23 questions from the slides · click a question for its answer

MCQ **Q1.** Which tool is specifically designed to transfer data in bulk between an RDBMS (e.g., MySQL) and HDFS?

(a) Apache Flume
(b) Apache Kafka
(c) Apache Sqoop
(d) Apache NiFi

(c) Apache Sqoop
Sqoop (SQL-to-Hadoop) bulk-copies tables between a database and HDFS; Flume is for logs.

MCQ **Q2.** Apache Kafka's data model is based on which fundamental storage abstraction?

(a) B-tree indexes for fast point lookups
(b) Column-family storage (like HBase)
(c) Append-only immutable distributed logs (partitioned topics)
(d) Document storage with secondary indexes

(c) Append-only immutable distributed logs (partitioned topics)
Kafka is a distributed, partitioned, replicated commit log.

MCQ **Q3.** In the Lambda architecture, which layer handles **real-time**, **low-latency** query views?

(a) Batch Layer
(b) Serving Layer
(c) Speed Layer
(d) Ingestion Layer

(c) Speed Layer
The Serving layer only merges views; the Speed layer computes the seconds-old views.

MCQ **Q4.** The Kappa architecture differs from the Lambda architecture primarily by:

(a) Adding a third processing layer for machine learning
(b) Using only relational databases for both batch and stream storage
(c) Eliminating the batch layer and using Kafka log replay for all reprocessing
(d) Replacing Hadoop with Spark in the batch layer while keeping the speed layer

(c) Eliminating the batch layer and using Kafka log replay for all reprocessing
Kappa removes the whole batch layer; (d) is the common misconception.

TRUE/FALSE **1.** Apache Sqoop is designed for real-time streaming ingestion from transactional systems.

False. Sqoop runs batch MapReduce imports. Corrected: Sqoop is designed for bulk batch transfer between an RDBMS and HDFS; real-time ingestion uses Kafka.

TRUE/FALSE **2.** Kafka retains messages for a configurable period, allowing consumers to replay events from any past offset.

True. Messages stay for the retention period (e.g. 7 days) after being read, so a consumer can reset its offset and replay.

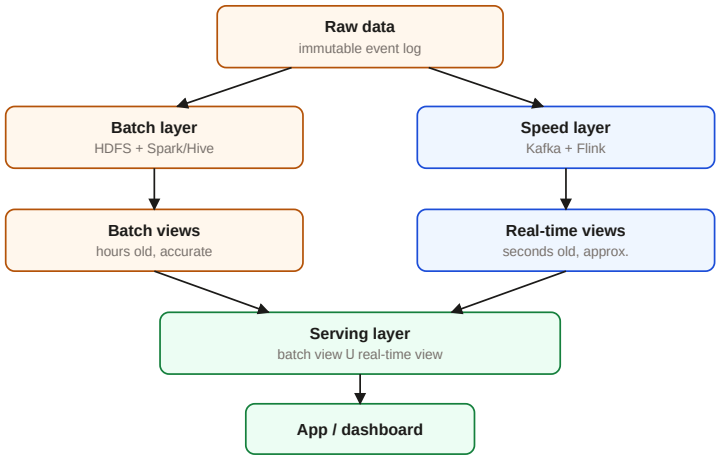
TRUE/FALSE **3.** The Lambda architecture requires maintaining two separate code paths implementing the same business logic.

True. The same logic is written for the batch layer (Spark/Hive) and the speed layer (Spark Streaming) and must be kept in sync.

TRUE/FALSE **4.** Apache Flume is the preferred tool for collecting and aggregating server log data into HDFS.

True. Flume moves logs into HDFS through Source, Channel and Sink with fan-in from many servers.

DESCRIPTIVE **D1.** Explain the **Lambda architecture** with a labelled diagram. What are its three layers? What is its primary engineering disadvantage compared to the Kappa architecture?



- **Batch:** master dataset on HDFS, recomputed hourly/nightly; accurate, 1–24 h stale.
- **Speed:** stream processing of data since the last batch; approximate, results within seconds.
- **Serving:** query = batch view UNION real-time view.

Disadvantage vs Kappa: two code paths for the same logic, kept in sync (30–40% of effort), plus two clusters and a runtime merge. Kappa has one stream code path.

DESCRIPTIVE **D2.** Describe how Apache Kafka achieves **high throughput and fault tolerance**. Explain the roles of topics, partitions, offsets, brokers, and replication factor in the Kafka data model.

Term	Role
Topic	Named append-only log for one event type
Partition	Ordered slice of a topic; P partitions run in parallel; same key → same partition
Offset	Message position; each consumer tracks its own
Broker	Server storing partitions
Replication factor R	1 leader + R − 1 followers per partition (default 3)

Throughput: sequential writes, zero-copy sendfile(), batching, partition parallelism.

Fault tolerance: a follower takes over when a leader's broker fails; retention lets a crashed consumer resume from its offset. Example: bkash-txns, 3 partitions, RF = 2 survives 1 broker failure.

DESCRIPTIVE **D3.** What is a **data lake**? How does it differ from a data warehouse in terms of schema enforcement, storage format, and typical use cases? What is a “data swamp” and how is it prevented?

Data lake: central store of raw data in any format, no upfront schema.

	Lake	Warehouse
Schema	On read	On write
Format	Raw files, any type	Cleaned relational tables
Use	ML, data science, exploration	BI reports, dashboards

Data swamp: a lake with no governance: no catalogue, stale vs current unknown, unversioned copies, PII mixed in.

Prevention: catalogue at ingest, owner per dataset, access control, quality checks, Delta Lake/Iceberg for ACID.

ANALYTICAL

A1. Pipeline Design for a Bank: Design a data integration pipeline for a Bangladeshi commercial bank that must:

- (a) Synchronise 10 million customer records nightly from Oracle to Hadoop for analytics
- (b) Detect fraudulent card-swipe events in real time with a decision within 200 ms
- (c) Power a risk management dashboard with data no older than 5 minutes

Specify which tool (Sqoop, Flume, Kafka, Spark Streaming) you would use at each stage. Justify each choice with reference to latency, throughput, and fault-tolerance requirements. Which architecture (Lambda or Kappa) fits best, and why?

Need	Tool	Latency	Throughput	Fault tolerance
(a) 10 M rows nightly	Sqoop (incremental import)	Hours is fine: batch	Parallel map tasks by key range, GB–TB/h	Failed map task retried; HDFS replicated
(b) Fraud < 200 ms	Kafka topic (keyed by card no.) + Spark Streaming	Milliseconds	Partitions scale consumers	RF = 3; restart from last offset
(c) Dashboard < 5 min	Second consumer group on same topic + Spark Streaming aggregates	Minutes	Same topic, no extra ingest	Replay from offset

Flume is not used: it is for logs, not tables or card events.

Architecture: Lambda. The bank has a real batch need (nightly sync, regulatory batch reports), which the slide lists as a reason Kappa is not suitable. Batch = Sqoop + HDFS + Hive; Speed = Kafka + Spark Streaming; Serving = dashboard. Cut the dual-path cost by making Kafka the single source both layers read.

ANALYTICAL

A2. Lambda to Kappa Migration: A team using Lambda architecture complains that maintaining two code paths (batch + stream) doubles development cost. Their CTO suggests migrating to Kappa. Analyse: (a) what must be true about the stream processing engine for Kappa to be viable; (b) what are the risks of migrating; (c) under what conditions is Kappa **not suitable** (give at least two concrete scenarios from the Bangladesh financial or healthcare sector)?

(a) Conditions: the engine reprocesses history about as fast as live data; full history sits in a replayable log (long Kafka retention or Iceberg on S3; default is only 7 days); the logic fits a stream job; exactly-once processing so replays match.

(b) Risks: TB+ replay may take days; batch throughput falls to stream-engine speed; storage cost of long retention; every Hive/Spark batch job must be rewritten and run in parallel during the switch; staff must learn stream SQL.

(c) Not suitable:

1. **Regulatory reconciliation:** Bangladesh Bank requires a daily end-of-day batch reconciliation; the batch layer must stay.
2. **TB+ history:** a national health-records system recomputing 10 years of disease statistics after a coding change; one Spark batch job is far faster than replay.
3. **Batch SQL preferred:** yearly credit-risk model training written in Hive/Spark SQL over full history.

Verdict: move streaming workloads to one Kappa path; keep batch where regulation or huge history demands it.

DISCUSSION

L1. Kafka as the single source of truth: all events written to Kafka first; the speed and batch layers are both consumers of the same topic — eliminating the data inconsistency problem of classic Lambda (two separate ingestion paths)

Claim: Netflix writes each event to Kafka once; Flink (speed, < 1 min) and the S3 sink + Spark/Iceberg (batch, ~1 day) read the same topic, so both layers see the same data. **For:** no mismatch between layers; new consumers just subscribe. **Against:** Kafka becomes a single critical dependency (2,000+ brokers to keep up), and the two code paths still exist; only ingestion is unified.

DISCUSSION

L2. Retention enables reprocessing: 7-day retention lets Netflix replay any event window when a Flink job has a bug or a Spark job fails

Claim: reading does not delete events, so a fixed job resets its offset and recomputes the last week. **For:** bugs are fixed after the fact without asking devices to resend. **Against:** only 7 days; a bug found on day 8 needs the S3/Iceberg copy. At 100 billion events/day, 7 days is 700 billion events of storage; tiered storage (Kafka 3.6) moves old segments to S3.

DISCUSSION **L3. Kappa simplification**: Netflix increasingly moved batch jobs to Flink stream processing (processing historical S3 data as a bounded stream), reducing pipeline complexity

Claim: a bounded stream (data with a start and an end, such as last month's S3 files) can be read by the same Flink code as live data, so one code path covers both. **For:** less duplicate code, fewer systems. **Against:** Netflix still keeps Spark + Iceberg for ML training and billing, where batch is easier or cheaper; the move is gradual.

DISCUSSION **L4. Schema Registry is non-negotiable at scale**: 4,000 topics × evolving schemas ⇒ consumer breakage without schema governance

Claim: a schema is a message's field names and types; if a producer renames a field, consumers break. Schema Registry stores the Avro/Protobuf schema per topic so consumers can read any past producer version. **For:** 4,000+ topics and 150+ Flink jobs cannot be coordinated by hand. **Against:** another service to run, and teams must follow compatibility rules. It is the streaming form of the data-swamp lesson.

DISCUSSION **L5. Backpressure is the real production problem**: during Super Bowl commercials, event rates spike 5×; Kafka's durable log absorbs the burst; Flink consumers catch up within minutes

Claim: backpressure is data arriving faster than consumers can process. If the 5× spike landed on the 1.3 M/s peak, that would be 6.5 M/s; Kafka appends the excess to disk and Flink catches up later. **For:** no data lost, no need to size consumers for the rare peak. **Against:** results lag by minutes during the spike (the 5-minute A/B latency slips), and brokers need spare disk.

LIVE EXERCISE **Task.** Simulate a Kafka producer/consumer pipeline using the **kafka-python** library with a local single-broker cluster (Docker Compose provided below).
Quick-start: `pip install kafka-python` ; `docker run -p 9092:9092 apache/kafka:3.7.0`

```
import json, random, time
from kafka import KafkaProducer, KafkaConsumer

TOPIC   = "play-events"
BROKER  = "localhost:9092"
CONTENT = [f"show_{i}" for i in range(20)]

# --- Producer ---
producer = KafkaProducer(
  bootstrap_servers=BROKER,
  value_serializer=lambda v:
      json.dumps(v).encode())

for _ in range(1_000):
    event = {
      "user_id":    random.randint(1, 500),
      "content_id": random.choice(CONTENT),
      "action":     random.choice(
                        ["play","pause","seek"]),
      "ts":         int(time.time())
    }
    producer.send(TOPIC, event)
producer.flush()
print("Produced 1,000 events")

# --- Consumer ---
consumer = KafkaConsumer(
  TOPIC,
  bootstrap_servers=BROKER,
  group_id="cse4345-group",
  auto_offset_reset="earliest",
  value_deserializer=lambda m:
      json.loads(m.decode()))

counts = {}
for msg in consumer:
    cid = msg.value["content_id"]
    counts[cid] = counts.get(cid, 0) + 1
    if sum(counts.values()) ⩾ 1_000:
        break
print(sorted(counts.items(),
             key=lambda x: -x[1])[:5])
```

The producer sends 1,000 JSON events (user 1–500, one of 20 shows, play/pause/seek); `flush()` waits until queued messages are sent. The consumer (group `cse4345-group` , `auto_offset_reset="earliest"`) reads from the start, counts events per `content_id` , stops after 1,000 and prints the top 5 shows. Broker: `docker run -p 9092:9092 apache/kafka:3.7.0` .

Replay (goal 3): `consumer.seek_to_beginning(*consumer.assignment())` . New group (goal 4): change `group_id` .

LIVE EXERCISE **1.** Run the producer — send 1,000 synthetic Netflix play events to topic `play-events` .

Expected output: `Produced 1,000 events` . With one partition the messages sit at offsets 0–999. Running it again appends offsets 1,000–1,999; nothing is overwritten.

LIVE EXERCISE **2.** Run the consumer — read all messages, count plays per `content_id` .

Expected: 1,000 events read. The slide's consumer counts every event per `content_id` (play, pause and seek alike), so each of the 20 shows gets about 1,000 / 20 = **50**; the top-5 print shows the largest. To count plays only, add `if msg.value["action"] == "play"` : about 1,000 / 3 ≈ 333 plays, about 16.7 per show. Exact counts change each run because the producer is random.

LIVE EXERCISE **3.** Reset the consumer offset to the *beginning* and re-consume — verify idempotent results.

After `seek_to_beginning` the same group re-reads all 1,000 events and gets identical counts. Reading did not delete anything, so the result is idempotent (same answer when repeated). If the producer ran again in between, there are 2,000 events: that is new data, not a broken replay.

LIVE EXERCISE **4.** Change the consumer group ID and observe that a new group starts from offset 0.

A new group has no committed offset, so with `auto_offset_reset="earliest"` it reads all 1,000 events from offset 0. The old group's offset is untouched. Trap: kafka-python's default is "latest", which makes a new group read 0 events.

CHAPTER 5 · WEEK 9

Data visualization

How to draw big data so people see the truth in it: Tufte's rules for clean, honest charts, Shneiderman's rule for interactive ones, and picking the chart from the question. Then the three problems scale adds (overplotting, latency, streaming), the tools that solve them, and the NYT COVID tracker as a worked case.

Exam question pool

19 questions from the slides · click a question for its answer

MCQ **Q1.** Edward Tufte's “data-ink ratio” principle states that a visualization should:

(a) Use more colours to highlight different categories of data
(b) Maximise the fraction of ink devoted to representing actual data, removing all decorative elements
(c) Use ink sparingly to reduce printing costs
(d) Always include data labels using a different ink colour from the chart body

(b) Maximise the fraction of ink devoted to representing actual data, removing all decorative elements
The ratio is about what the ink encodes, not saving ink.

MCQ **Q2.** Which visualization type is most appropriate for showing the distribution and spread of a continuous variable?

(a) Pie chart
(b) Line chart
(c) Histogram
(d) Treemap

(c) Histogram
Bins along the number line show shape and spread.

MCQ **Q3.** Shneiderman's visualization mantra begins with which step?

(a) Details first, then overview
(b) Overview first, zoom and filter, then details on demand
(c) Filter first, then provide an overview
(d) Details on demand first, then zoom out

(b) Overview first, zoom and filter, then details on demand
Filtering only makes sense after the overview shows what to filter.

MCQ **Q4.** Which visualization tool is best suited for a real-time production operations dashboard monitoring server metrics and application error rates with 5-second refresh intervals?

(a) Tableau
(b) D3.js
(c) Power BI
(d) Grafana

(d) Grafana
Native time-series DB integration with 5 s auto-refresh; D3 would need custom code.

TRUE/FALSE **1.** A pie chart is a recommended choice for comparing more than 5 categories.

False. People judge angles poorly. Corrected: use a horizontal bar chart; pies only for ≤ 4 categories.

TRUE/FALSE **2.** Hexbin plots are an effective technique for reducing overplotting when visualising millions of data points.

True. Counting points per hexagon turns 10 million dots into ≈ 10,000 coloured cells.

TRUE/FALSE **3.** D3.js generates static PNG or PDF image files as its primary output format.

False. Corrected: D3.js binds data to DOM elements and produces interactive SVG in the browser.

TRUE/FALSE **4.** Choropleth maps are used to visualise the geographic distribution of a quantitative variable.

True. Regions are shaded by a value, best a rate such as cases per 100,000.

DESCRIPTIVE **D1.** Explain Tufte's data-ink ratio principle. Provide a “before and after” example: describe a cluttered bar chart (what chart-junk elements are present) and then describe the clean version after applying the principle.

Principle: data-ink ratio = ink encoding data ÷ total ink; maximise it. If erasing an element loses no information, erase it.

Before (quarterly sales)	After
Title "CHART TITLE!!!"	Plain short title
Legend for one series	No legend; Q1-Q4 on the x-axis
3-D bars with shadows	Flat 2-D bars, one colour
Gradient fill, border	Removed
Heavy dark gridlines	None, or one light line; y starts at 0

Same data, but almost all remaining ink is data, so the trend reads at once.

DESCRIPTIVE **D2.** What is overplotting? Describe three techniques to address it when visualising millions of data points in a scatterplot. For each technique, explain what information it preserves that the original overplotted chart lost.

Overplotting: millions of dots overlap into a blob that only says "data exists here".

Technique	How	Preserves
Alpha transparency	Each point α = 0.02; stacks look darker	Relative density (fails at very high density)
Hexbin	Count points per hexagon, colour by count; 10 M → ≈ 10,000 cells	Actual count per area
KDE contour	Smooth density, contour lines	Shape, modes (peaks), tails

DESCRIPTIVE **D3.** What is a data cube? How does pre-aggregating a billion-row fact table into a data cube enable interactive visualization at sub-second latency? What is the trade-off of this approach?

Data cube: pre-computed aggregates (SUM, COUNT) of a fact table for combinations of dimensions, e.g. district × date × product × channel.

- A nightly Spark/Hive job builds it once.
- It holds millions of rows, not billions.
- Tableau/Superset filters read the cube, so they answer in under 1 s instead of minutes.

Trade-off: staleness up to 24 h; only pre-built dimensions can be sliced; extra storage. Alternative: AQP (sampling) trades accuracy for speed.

ANALYTICAL **A1.** Public Health Dashboard Design: A DGHS public health dashboard needs to communicate: (a) COVID-19 confirmed case counts by district over 24 months (b) Real-time hospitalisation rate updated every 5 minutes (c) Demographic breakdown (age group, gender) of confirmed cases For each sub-requirement: specify the appropriate visualization type, justify using Shneiderman's data-type taxonomy, identify which big data visualization challenge applies (if any), and name the tool you would use. Apply Tufte's data-ink principle to state one specific chart-junk element to eliminate.

Req.	Data type	Chart	Challenge	Tool
(a) Cases by district, 24 months	Temporal (+ 2-D map)	Small multiples of line charts, same scale; choropleth of rate per 100,000	Latency: pre-aggregate to daily counts per district	Tableau / Power BI
(b) Hospitalisation every 5 min	Temporal (live)	Stat panel + sparkline, 24 h line chart	Streaming	Grafana on InfluxDB/Prometheus
(c) Age × gender	Categorical / multi-dimensional	Grouped horizontal bar	None (small table)	Tableau / Power BI

Sizes: (a) 730 days × 64 districts = **46,720** rows after aggregation. (b) 24 × 60 ÷ 5 = **288** points per day, so 5 s polling is ample.

Mantra: national total → district dropdown and date slider → hover tooltip.

Chartjunk to remove: 3-D effect on the demographic bars (also heavy gridlines on the line charts).

ANALYTICAL

A2. Streaming Sentiment Visualization: Design a complete visualization system for a streaming Twitter (X) sentiment analysis pipeline processing 10,000 tweets per second. Specify: (a) which Kafka aggregation window you would use and why; (b) which chart types best convey sentiment drift over time and topic clustering; (c) how you would handle the velocity challenge at the visualization layer without overloading the browser; (d) which tools form the pipeline from Kafka to browser render.

Givens: 10,000 tweets/s → **600,000** per minute, **864,000,000** per day.

(a) 1-minute tumbling window per topic: count, mean sentiment, % positive/neutral/negative. One point replaces 600,000 tweets, smooths noise, still shows drift. Optional 15-min sliding window for a smoother trend.

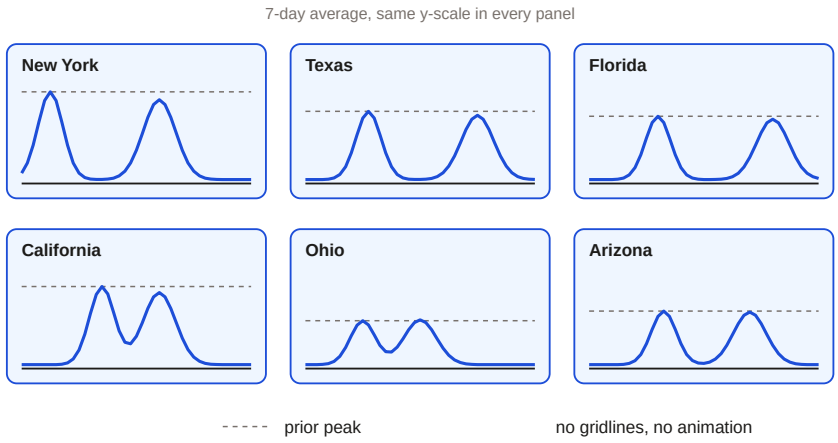
(b) Drift: line chart (or 100% stacked area) per topic as small multiples. Clustering: t-SNE/UMAP scatter drawn as hexbin or KDE to avoid overplotting.

(c) Send only window aggregates (one refresh of 5 s replaces 50,000 tweets); keep a rolling buffer of the last 60 points; throttle redraws; pre-bin the cluster view server-side.

(d) X API → Kafka → Spark Structured Streaming/Flink → InfluxDB/Prometheus or Elasticsearch → Grafana (5 s poll) or Kibana; WebSocket → D3.js for sub-second views.

DISCUSSION

Lesson 1. Data quality is a visualisation problem: the NYT's biggest engineering effort was cleaning data, not drawing charts; retroactive state corrections forced D3 charts to support negative bars (days where cases were revised down)



Illustrative NYT-style small multiples: one 7-day-average chart per state on a shared y-scale, no gridlines, one light reference line at the prior peak.

Claim: a chart is only as right as its data.

For: most NYT work was normalising 50 state schemas; downward revisions made daily changes negative, so chart design had to show negative bars.

Against: cleaning is really a data-engineering task, and no chart design can fix bad source data. Even so, the chart must show corrections honestly (negative bars) rather than hide them.

DISCUSSION

Lesson 2. Accessibility is non-negotiable at 6M readers/day: every chart had a data table alternative for screen readers; colour scales were tested for colour-blindness (deuteranopia, protanopia)

Claim: at 6 M readers a day, even a small share of blind or colour-blind readers is many people.

For: a screen reader cannot describe the shapes of a chart, so a data table gives blind readers the same facts; avoiding red/green scales serves red-green colour-blind readers of public-health data.

Against: tables and testing cost time on daily deadlines and restrict palettes; the NYT judged it worth paying.

DISCUSSION **Lesson 3.** Small multiples beat animation: the 50-state grid (Tufte small multiples) was more actionable than an animated map of spreading cases

Claim: showing every state at once beats one frame at a time.

For: a shared scale lets readers compare states side by side and find their own without waiting or remembering frames.

Against: animation grabs attention and shows geographic spread vividly; 50 small charts can be dense on a phone. Grid for analysis, animation for a headline.

DISCUSSION **Lesson 4.** Rolling averages are editorial decisions: choosing 7-day vs. 14-day average changes what story the chart tells; these are journalistic choices with ethical weight

Claim: the window length shapes the story readers see.

For 7-day: removes weekend reporting dips and reacts to a real turn within days.

For 14-day: smoother, fewer false alarms, but hides a turning point longer.

Readers made health decisions from it, so the choice should be stated on the chart.

DISCUSSION **Lesson 5.** GitHub as data infrastructure: publishing the raw CSV on GitHub enabled 1,000+ derivative dashboards worldwide within days

Claim: open data multiplies the value of expensive cleaning.

For: others reused it for 1,000+ dashboards; version history makes corrections traceable and numbers checkable.

Against: GitHub is not a database; others can misread the data, and the publisher must keep files consistent as corrections arrive.

LIVE EXERCISE

Task. Live Exercise — D3.js Bar Chart with Tufte Styling Task (25 minutes, pairs) Build a minimal D3.js bar chart that follows Tufte's data-ink principles: 1. No gridlines (or only a single light reference line) 2. y-axis starts at 0 3. No fill gradient — use a flat colour 4. Tooltip on hover (Shneiderman: details-on-demand) 5. Add a sparkline for each bar showing the 7-day trend Dataset: weekly Kafka message counts for 5 topics (generate 7×5 random values). Extension: add a dropdown filter to select one topic and zoom into its 7-day line chart.

```
<!DOCTYPE html>
<meta charset="utf-8">
<style>
  .bar { fill: steelblue; }
  .bar:hover { fill: #b5651d; }
  .axis path, .axis line {
    stroke: #ccc; stroke-width: 0.5px; }
  .tooltip {
    position: absolute; background: #fff;
    border: 1px solid #ccc; padding: 6px;
    font-size: 11px; pointer-events: none; }
</style>
<body>
<script src=
  "https://d3js.org/d3.v7.min.js"></script>
<script>
const topics = ["orders","clicks",
                "errors","logins","search"];
const data = topics.map(t => ({
  topic: t,
  count: Math.floor(Math.random()*50000)
         + 10000
}));
const svg = d3.select("body").append("svg")
  .attr("width", 500).attr("height", 300);
const x = d3.scaleBand()
  .domain(data.map(d=>d.topic))
  .range([50,480]).padding(0.3);
const y = d3.scaleLinear()
  .domain([0, d3.max(data,d=>d.count)])
  .range([260, 20]);
svg.append("g").attr("transform","translate(0,260)")
  .call(d3.axisBottom(x));
svg.append("g").attr("transform","translate(50,0)")
  .call(d3.axisLeft(y).ticks(4));
svg.selectAll(".bar").data(data).join("rect")
  .attr("class","bar")
  .attr("x",d=>x(d.topic))
  .attr("y",d=>y(d.count))
  .attr("width",x.bandwidth())
  .attr("height",d=>260-y(d.count));
</script>
```

Key code (added to the skeleton; 7 daily values per topic, bar = weekly sum; SVG height raised to 340 for the sparklines; CSS `.ref { stroke: #ddd; stroke-dasharray: 3 3; }`):

```
const data = topics.map(t => { const days = d3.range(7)
  .map(() => Math.floor(Math.random()*50000) + 10000);
  return {topic: t, days, count: d3.sum(days)}; });
svg.selectAll("g").attr("class","axis");   // skeleton forgot this
const mean = d3.mean(data, d => d.count);  // one light line at the mean
svg.append("line").attr("class","ref").attr("x1",50).attr("x2",480).attr("y1",y(mean)).attr("y2",y(mean));
const tip = d3.select("body").append("div").attr("class","tooltip").style("opacity",0);
svg.selectAll(".bar").on("mouseover",(e,d)⇒tip.style("opacity",1)
      .style("left",(e.pageX+8)+"px").style("top",(e.pageY-28)+"px")
      .html(d.topic+": "+d.count))
    .on("mouseout",()⇒tip.style("opacity",0));
const sx = d3.scaleLinear().domain([0,6]).range([0,x.bandwidth()]);
data.forEach(d => { const sy = d3.scaleLinear()
  .domain(d3.extent(d.days)).range([325,290]);
```

```
svg.append("path").attr("fill","none").attr("stroke","#555").attr("transform",`translate(${x(d.topic)},0)`)
   .attr("d", d3.line().x((v,i)⇒sx(i)).y(v⇒sy(v))(d.days)); });
```

1. **Gridlines:** D3 axes draw none; add only one dashed light line at the mean.
2. **y from 0:** domain `[0, max]` ; a higher start would push the lie factor above 1.
3. **Flat colour:** `fill: steelblue` , no gradient.
4. **Tooltip:** mouseover shows topic and count; mouseout hides it. This is details on demand.
5. **Sparkline:** 7-point line, no axes, under each bar, own min-max scale (shape, not size).

Extension: a `select` of the 5 topics; on change, fade other bars and draw that topic's 7-day line chart below.

Expected: 5 flat bars; tallest reaches y = 20 (240 px); bar width ≈ 56.8 px (step 430 ÷ 5.3 ≈ 81.1); weekly totals between 70,000 and 419,993.

CHAPTER 6 · WEEK 10

Big data in real applications

This chapter applies the course's stack (Kafka, Spark, HDFS, Hive, Grafana) to three domains: social media graphs, time series forecasting and healthcare data. A flu surveillance case study shows what big data signals can and cannot replace.

Exam question pool

22 questions from the slides · click a question for its answer

MCQ **Q1.** Facebook's Prophet library is primarily designed for:

(a) Graph analytics and community detection in social networks
(b) Real-time stream processing of events
(c) Time series forecasting with decomposable trend, seasonality, and holiday components
(d) Text sentiment classification using deep learning

(c) Time series forecasting with decomposable trend, seasonality, and holiday components
Prophet fits $y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$.

MCQ **Q2.** Which algorithm is most commonly used for community detection in large-scale social network graphs?

(a) K-Means clustering
(b) PageRank
(c) Louvain algorithm
(d) ARIMA

(c) Louvain algorithm
It maximises modularity in O(n log n); PageRank ranks importance, not groups.

MCQ **Q3.** HIPAA in the United States primarily regulates the privacy and security of:

(a) Financial transaction data and bank records
(b) Healthcare data including patient records and medical information
(c) Social media user data and behavioural profiles
(d) Government data retention and public records

(b) Healthcare data including patient records and medical information
HIPAA is the Health Insurance Portability and Accountability Act; financial data falls under GLBA.

MCQ **Q4.** DeepMind's AlphaFold used big data and deep learning to solve:

(a) Real-time financial fraud detection at bank-card transaction scale
(b) Social network influence maximisation over billion-node graphs
(c) The protein structure prediction problem, predicting 3D structure from amino acid sequence
(d) Time series anomaly detection in IoT sensor streams

(c) The protein structure prediction problem, predicting 3D structure from amino acid sequence
Trained on 170,000 PDB structures with MSAs from 250M sequences; 2024 Nobel Prize in Chemistry.

TRUE/FALSE **1.** Google Flu Trends successfully replaced traditional CDC flu surveillance and has been used as the primary source ever since.

False. GFT over-predicted flu by 2× in 2012-2013 and was discontinued.
Corrected: Google Flu Trends failed, and CDC still relies on clinical reporting, with big data signals only as a supplement.

TRUE/FALSE **2.** Time series data is order-sensitive: randomly shuffling the records in a time series changes the analysis result.

True. Autocorrelation, trend and seasonality exist only in time order; shuffling destroys them.

TRUE/FALSE **3.** Social network graphs are typically sparse, with most nodes connected to only a small fraction of all possible neighbours.

True. Real social networks are scale-free (power-law degrees): most nodes have few links and a few hubs have millions. Twitter

had 1.47B edges among 41.7M users, about 35 follows each out of millions possible.

TRUE/FALSE **4.** Medical imaging data such as MRI and CT scans is classified as structured data because it is stored in files.

False. Being in a file does not make data structured.
Corrected: MRI and CT scans are unstructured binary DICOM files (200 MB to 2 GB), analysed by deep learning, not SQL.

DESCRIPTIVE **D1.** Describe three types of analysis commonly performed on social media data (e.g., community detection, sentiment analysis, influence maximisation). For each, state one specific ethical risk that arises when the analysis is deployed at scale.

Analysis	What it does	Ethical risk at scale
Community detection	Louvain finds groups more densely linked inside than outside, by maximising modularity.	Profiling: group labels expose political or religious ties users never shared.
Sentiment analysis	VADER scores each post in [−1, +1]; > 0.05 positive, < −0.05 negative.	Suppressing dissent: flagging people who criticise a government policy.
Influence maximisation	Picks k seed users for widest spread; greedy gives ≥ 63% of optimal.	Manipulation: spreading misinformation or microtargeting vulnerable users.

Safeguard for all three: transparency and purpose limitation.

DESCRIPTIVE **D2.** What is the difference between ARIMA and Facebook Prophet for time series forecasting? Under what conditions would you prefer Prophet over ARIMA? When might ARIMA still be the better choice?

ARIMA(p, d, q) predicts from past p values and past q errors after differencing d times to make the series stationary. **Prophet** adds up trend with changepoints, Fourier seasonality and a holiday term: $y = g + s + h + \varepsilon$.

Prefer Prophet when the series has several seasonalities, holiday effects (Eid, Puja), trend shifts, missing data or outliers, or when thousands of series must be fitted automatically (5,000 SKUs in 12 minutes with applyInPandas).

ARIMA is still better for a short, stationary series with no strong seasonality and little data, where an expert can tune one important series closely.

DESCRIPTIVE **D3.** Identify three data integration and privacy challenges specific to healthcare big data analytics. For each, describe one technical mitigation strategy (e.g., differential privacy, federated learning, de-identification).

Challenge	Mitigation
Records carry names, DOB, NID, phone; HIPAA bars research use as-is	**De-identification:** safe harbour removes the 18 identifiers, or expert determination certifies low risk
Models need 1M+ records, but data cannot leave each hospital (consent, PDPA)	**Federated learning:** train at each hospital, share only model updates
Published counts for small areas can reveal one patient	**Differential privacy:** add calibrated noise to aggregates

Integration itself: FHIR, Parquet genomics, Kafka wearables and DICOM images meet in one HDFS/S3 lake with a Hive Metastore.

ANALYTICAL **A1.** Flu Outbreak Early-Warning System for Bangladesh: Design a big data system to detect flu outbreaks in Bangladesh using Twitter/Facebook posts and pharmacy purchase records from major pharmacy chains. (a) How would you collect and ingest these two data sources? (Specify tools: Kafka, Sqoop, Flume, REST API) (b) What features would you extract from each source for a prediction model? (Name specific NLP features and pharmacy signal metrics) (c) What model would you use, and how would you validate it against ground-truth CDC/DGHS surveillance data? (d) What are the ethical risks of this system? How would you mitigate them?

Design: sources → Kafka → Spark Streaming → alert engine + Grafana (the CDC pattern).

Part	Social posts	Pharmacy sales
(a) Ingest	REST APIs (X API v2, Facebook Pages), keyword filter in Bangla and English, producer writes to Kafka topic `social_raw`	Sqoop for nightly bulk import from the chains' databases; Flume ships POS logs to Kafka topic `pharmacy_sales`

(b) Features	Daily flu-keyword post count per district; share of symptom terms (fever, cough, 'jor'); first-person illness flag; VADER/Bangla sentiment; ratio to 4-week baseline	Daily paracetamol and cough-syrup units per district; week-on-week % change; z-score vs same week last year

(c) Model: Prophet per district on DGHS weekly ILI counts, yearly seasonality, Eid holidays, social and pharmacy features as extra regressors; run districts in parallel with applyInPandas. **Validate** with a rolling backtest against DGHS counts: error (MAE), lead time, hit rate, false alarms, and test on an unseen season (GFT had $r^2 = 0.97$ in-sample and still failed).

(d) Ethics: privacy (keep district counts only, drop user IDs, add differential-privacy noise); panic from false alarms (an officer confirms before any announcement); bias against rural and older people who post less (weight with pharmacy and DGHS data); misuse for watching individuals (purpose limitation, access logs, PDPA consent).

ANALYTICAL

A2. Retail Demand Forecasting Pipeline: A large Bangladeshi supermarket chain needs 90-day demand forecasts for 5,000 store–product SKU combinations. Design the end-to-end big data pipeline: (a) data ingestion from POS systems; (b) storage and partitioning strategy in HDFS/S3; (c) forecasting model choice (ARIMA vs. Prophet vs. LSTM) and justification; (d) how you parallelise model fitting across 5,000 SKUs using Spark; (e) how forecasts are served to the inventory management dashboard. Identify the primary bottleneck in your pipeline.

Stage	Design
(a) Ingest	Tills send sale events to Kafka topic `pos_sales`; Sqoop bulk-loads history; Spark aggregates to (sku_id, date, units)
(b) Storage	Raw Parquet on HDFS/S3 partitioned by date; daily aggregate Hive table partitioned by date, bucketed by sku_id; forecasts partitioned by run_date
(c) Model	**Prophet:** weekly and yearly seasonality, holidays (Eid, Puja, Pohela Boishakh), automatic changepoints, tolerates gaps. ARIMA needs hand tuning per series; LSTM needs much data, GPUs and is a black box
(d) Parallel fit	`daily.groupBy("sku_id").applyInPandas(fit, schema)`; each group goes to one task, Prophet fits inside the Pandas UDF
(e) Serve	Write Parquet to Hive/S3; load latest run into a serving table; dashboard (Superset/Grafana) shows yhat with bands and reorder alerts

Numbers: 8 h = 480 min sequential; 12 min on 20 executors; speedup = 480 / 12 = **40×**; 5.76 s per model; 250 models per executor.

Bottleneck: model fitting (Prophet via Stan, the 8-hour step). Skewed SKUs with long histories finish last; cap history and add cores.

DISCUSSION

1. Kwak et al. found Twitter's average shortest path is 4.12 hops over 41.7M users. If a misinformation tweet is retweeted by a high-PageRank account, estimate how many users it reaches within 3 hops assuming an average out-degree of 45. What graph property makes this estimate an overestimate? [CLO4 — Apply]

Estimate: $45 + 45^2 + 45^3 = 45 + 2{,}025 + 91{,}125 =$ **93,195 users** (0.22% of 41.7M).

For an overestimate: clustering. Followers overlap, so the tree counts the same people again; most viewers also do not retweet.

Against: a high-PageRank account may have over a million followers, so hop 1 alone can beat 45.

DISCUSSION

2. A Prophet model for weekly hospital admissions has strong annual and weekly seasonality but misses a sudden COVID-19 wave onset. Identify which Prophet component should detect this changepoint and explain what prior hyperparameter controls the model's sensitivity to new changepoints. [CLO4 — Analyse]

Component: the trend g(t). A wave is a structural break, not a repeating cycle or a listed holiday.

Hyperparameter: `changepoint_prior_scale`, the scale of the sparse prior on the slope changes δ. Default 0.05 keeps the trend stiff and smooths over the wave; raising it (e.g. 0.5) lets the trend bend.

Against raising it: the trend then chases noise and the forecast band widens, so tune on held-out weeks.

DISCUSSION **3.** Design a dengue fever surveillance system for Bangladesh using BioSense-style ED data, Twitter signals, and Prophet forecasting. Specify: data sources, pipeline architecture (Kafka + Spark), model inputs, and how you would validate the system before policy use. [CLO4 — Design]

- **Sources:** ED visits (FHIR) as main signal, NS1 lab results, dengue tweets as leading signal, rainfall.
- **Pipeline:** Kafka topics → Spark Streaming de-identifies and aggregates per district → Parquet → nightly Prophet per district via applyInPandas → alerts + Grafana.
- **Inputs:** weekly dengue-like visits, yearly (monsoon) seasonality, Eid holidays, tweets and rainfall as regressors.
- **Validate:** backtest on past seasons and an unseen season; compare with and without Twitter.

For: earlier warning. **Against:** tweets skew urban and young, so ED data stays primary.

LIVE EXERCISE **Task.** Live Exercise — Prophet Flu Forecasting in Python. Task (25 minutes, pairs): Fit a Prophet model to synthetic weekly flu incidence data and produce a 12-week forecast. Install: pip install prophet matplotlib pandas

```
import pandas as pd
import numpy as np
from prophet import Prophet
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
dates = pd.date_range("2021-01-04",
                      periods=156, freq="W")
# Annual flu seasonality + noise
ili = (5 + 3*np.sin(
          2*np.pi*np.arange(156)/52
          - np.pi/2)
       + rng.normal(0, 0.5, 156)).clip(0)
df = pd.DataFrame({"ds": dates, "y": ili})

# Optional: add Eid holiday
holidays = pd.DataFrame({
   "holiday": "Eid",
   "ds":       pd.to_datetime([
                 "2021-05-13","2022-05-03",
                 "2023-04-22"]),
   "lower_window": 0,
   "upper_window": 3,
})

m = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
    changepoint_prior_scale=0.05,
    holidays=holidays)
m.fit(df)
future = m.make_future_dataframe(
    periods=12, freq="W")
forecast = m.predict(future)
m.plot(forecast)
m.plot_components(forecast)
plt.show()
```

Run the skeleton, then refit with the changes below.

```
m = Prophet(yearly_seasonality=True,
            weekly_seasonality=False,
            changepoint_prior_scale=0.05,   # try 0.5
            holidays=holidays,
            interval_width=0.95)            # 95% band
m.fit(df)
fc = m.predict(m.make_future_dataframe(periods=12, freq="W"))
m.plot(fc); m.plot_components(fc)
```

Expected: dates run 2021-01-10 to 2023-12-31 (weeks end Sunday); the forecast covers 2024-01-07 to 2024-03-24 and rises from the yearly low (true curve 2.0) towards about 4.3.

LIVE EXERCISE **1.** Generate 3 years of synthetic weekly ILI data with annual seasonality + noise.

The skeleton does it: 156 weekly dates (156 / 52 = 3 years), a sine with period 52 weeks running from 2 to 8 around a mean of 5, plus noise with SD 0.5. Columns must be named `ds` and `y`.

LIVE EXERCISE **2.** Fit Prophet with yearly and weekly seasonality.

`yearly_seasonality=True` fits the annual Fourier wave (P = 365.25). Keep `weekly_seasonality=False`: with one point per week, always a Sunday, there is no within-week pattern to learn.

LIVE EXERCISE **3.** Plot the forecast with 95% uncertainty intervals.

Pass `interval_width=0.95` (default is 80%), then `m.plot(forecast)`. Expected: data dots, the yhat line and a band from yhat_lower to yhat_upper that widens into the future.

LIVE EXERCISE **4.** Add a holiday for “Eid” (spike in healthcare visits) and observe the effect.

Expected effect is near zero: the synthetic data has no Eid spike, and the 2022 window (05-03 to 05-06) contains no Sunday, so it matches no point. Add +3 to the Eid weeks and the components plot shows an Eid bar near +3.

LIVE EXERCISE **5.** Change changepoint_prior_scale from 0.05 to 0.5 and compare trend flexibility.

At 0.05 the trend stays nearly flat around 5 (the true trend). At 0.5 it wiggles, absorbs noise and part of the seasonal wave, and the future band widens. Higher reacts faster to real breaks but overfits stable data.

LAST TEN MINUTES

Formula and fact sheet

Formulas

What	Formula	Note
W5 · Columnar data read	`Read = D × k / N`	D table size, k columns selected, N total columns
W5 · Columnar I/O reduction	`Reduction = 1 - k / N; speedup ≈ N / k`	10 TB, 3 of 100: 97%, 33×
W5 · Pushdown saving	`Speedup = D / R`	R = bytes actually read; 10 TB / 50 GB = 200×
W5 · Mappers per file	`Splits = ⌈F / B⌉, or 1 if gzip`	B = block or row group, about 128 MB
W5 · Containers per queue	`min(nodes·f·RAM / c_ram, nodes·f·CPU / c_cpu)`	smaller term is the bottleneck
W5 · Nodes for a job	`⌈T / min(RAM / c_ram, CPU / c_cpu)⌉`	T containers needed at once
W5 · Extra work from utilisation	`U_new / U_old - 1`	78/45 − 1 = 73%
W6 · MapReduce iterative disk I/O	`MR_IO = k × (D_read + D_write)`	k iterations; slide: 100 × (50 + 50) GB = 10 TB
W6 · Spark HDFS I/O	`Spark_IO = D (read once)`	later iterations read RAM, not HDFS
W6 · Transfer time	`t = size / speed`	50,000 MB / 500 MB/s = 100 s
W6 · Iterative wall-clock time	`T_MR = k × (D/R_hdfs + D/W_hdfs); T_Spark = D/R_hdfs + (k - 1) × D/R_ram`	k − 1 RAM passes, not k
W6 · Speedup	`Speedup = T_old / T_new`	A1: 20,000 / 1,090 ≈ 18.3×; RDD paper 127 / 0.96 ≈ 132×
W6 · Hash partitioner	`partition = key.hashCode() % numPartitions`	17 % 4 = 1
W6 · Shuffle records	`groupByKey = all pairs; reduceByKey ≤ partitions × distinct keys per partition`	4,000 vs 200 records
W6 · Efficiency per node	`(T_old × N_old) / (T_new × N_new)`	GraySort (72 × 2,100) / (23 × 206) ≈ 32×
W7 · Map-side aggregation	`reduction = N_rows / G (with M mappers: N_rows / (M × G))`	50M rows, 200 countries: 250,000×
W7 · Partition pruning	`fraction read = P_read / P_total`	P_total = product of values per partition column
W7 · Bucket number	`bucket = hash(column) % N`	integer key: hash = value; 98 % 4 = 2
W7 · Bucket file size	`size = S / (P × N); files = P × N`	aim near the 128 MB HDFS block
W7 · ALS model size	`(users + items) × k × bytes`	(75M + 30M) × 50 × 4 B = 21 GB
W7 · Speed-up	`T_old / T_new; saved = (T_old - T_new) / T_old`	8 h to 2 h: 4×, 75% saved
W7 · Pairs in a playlist	`L(L - 1) / 2`	20 songs: 190 pairs
W8 · Point-to-point ETL jobs	`N × (N - 1); bus = N`	50 systems: 2,450 jobs vs 50 connections
W8 · Partition copies	`P × R`	bkash-txns: 3 × 2 = 6
W8 · Broker failures survived	`R - 1`	default R = 3 survives 2

W8 · Active consumers in a group	`min(P, C); idle = C - P (only if C > P)`	one consumer per partition per group
W8 · Messages per second	`MB/s × 1,000,000 ÷ message bytes`	50.5 MB/s, 100 B = 505,000 msgs/s
W8 · Speedup	`Kafka ÷ other`	50.5 / 4.3 ≈ 11.7×; 50.5 / 14.7 ≈ 3.4×
W8 · Sequential vs random disk	`seq ÷ random`	600 / 100 = 6× (paper figures)
W8 · Zero-copy	`4 copies → 2 copies`	about 2× throughput, CPU near zero
W8 · Events per second	`events per day ÷ 86,400`	Netflix 100 B/day ≈ 1.16 M/s
W9 · Data-ink ratio	`ink encoding data ÷ total ink`	Maximise it; 1 is the ceiling
W9 · Lie factor	`effect shown in graphic ÷ effect in data`	1 truthful; above 1 overstates; below 1 understates
W9 · Hexbin reduction	`points ÷ hex cells`	10 M ÷ 10,000 = 1,000×
W9 · Alpha stacking	`opacity = 1 - (1 - α)^n`	α = 0.02: n = 100 → 0.867, saturates
W9 · Cube rows	`≤ $d_1 \times d_2 \times \dots \times d_k$; reduction = fact rows ÷ cube rows`	100 B ÷ 5 M = 20,000×
W9 · Group-by combinations	`$2^k$ for k dimensions`	k = 4 → 16
W9 · Events per window	`rate × window seconds`	10,000/s × 60 = 600,000
W9 · Updates per day	`86,400 ÷ refresh seconds`	Every 5 min → 288
W10 · Prophet model	`y(t) = g(t) + s(t) + h(t) + ε(t)`	trend + seasonality + holidays + noise
W10 · Prophet trend	`g(t) = (k + a(t)^T δ) t + (m + a(t)^T γ)`	δ slope changes at changepoints, sparse prior
W10 · Fourier seasonality	`$s(t) = \Sigma [a_n \cos(2\pi n t/P) + b_n \sin(2\pi n t/P)]$, n = 1..N`	P = 365.25 yearly, 7 weekly
W10 · ARIMA(p, d, q)	`AR past p values; I difference d times; MA past q errors`	needs stationary input
W10 · Speedup	`T_sequential / T_parallel`	480 min / 12 min = 40× (5,000 SKUs)
W10 · Reach in h hops	`d + d² + … + d^h`	upper bound; d = 45, h = 3 gives 93,195
W10 · Greedy influence bound	`spread ≥ (1 - 1/e) × optimal ≈ 0.632 × optimal`	Kempe et al. 2003
W10 · Modularity (words)	`fraction of edges inside communities - fraction expected by chance`	Louvain maximises it
W10 · Power law	`P(k) ∝ k^(-α), α ≈ 2.276`	Twitter follower counts (Kwak 2010)
W10 · Spike ratio	`current rate / baseline rate`	10 to 10,000 per hour = 1,000×
W10 · VADER classes	`> 0.05 positive; < -0.05 negative; else neutral`	compound in [−1, +1]
W10 · STL anomaly rule	`|residual| > k × IQR, IQR = Q3 - Q1`	IQR of residuals, not raw data
W10 · Genomics storage	`200 GB × patients (× replication)`	10,000 patients = 2 PB

Every topic in one line

Wk	Topic	One-line answer
W5	**YARN and its three components**	YARN is Hadoop 2's resource manager: one ResourceManager gives out containers, a NodeManager runs them on each node, and one ApplicationMaster manages each job.
W5	**YARN fault tolerance**	When a NodeManager stops heartbeating, the RM marks its containers failed and the AM requests replacements on other nodes.
W5	**Schedulers and capacity planning**	The Capacity Scheduler splits the cluster into queues with guaranteed shares and lets queues borrow idle capacity.
W5	**Row vs columnar storage and Parquet**	Columnar storage keeps each column's values together on disk, so a query reads only the columns it needs.
W5	**File formats: Avro, Parquet, ORC**	Avro is a row format built for schema evolution; Parquet and ORC are columnar formats built for analytics.
W5	**Compression codecs and splittability**	A splittable file can be read from any point, so multiple Mappers process different parts at the same time.
W6	**Why Spark: the MapReduce iteration bottleneck**	MapReduce writes every intermediate result to HDFS, so iterative algorithms pay disk I/O on every pass; Spark caches the data in RAM once.
W6	**Spark architecture**	Spark has three tiers: a Driver that builds the DAG and schedules tasks, a Cluster Manager that allocates resources, and Executors that run tasks and cache data.
W6	**RDDs and lineage-based fault tolerance**	An RDD is a read-only, partitioned collection of records that Spark rebuilds from its lineage if a partition is lost.
W6	**Transformations, actions and lazy evaluation**	Lazy evaluation means Spark records transformations in a DAG and runs nothing until an action is called.
W6	**Narrow vs wide dependencies, stages and shuffle**	A wide dependency is one where an output partition needs data from many input partitions, which forces a shuffle and a new stage.
W6	**DataFrames, Spark SQL and the ecosystem**	A DataFrame is a distributed table with named columns and a schema, optimised by Catalyst.
W7	**Hive architecture and query compilation**	Hive is a data warehouse layer on Hadoop that compiles SQL-like HiveQL queries into MapReduce, Tez or Spark jobs over files on HDFS.
W7	**Schema-on-read and HiveQL vs SQL**	Schema-on-read means Hive applies the table's column types when data is queried, not when it is loaded.
W7	**Partitioning and partition pruning**	Partitioning stores a Hive table in separate HDFS sub-directories by the value of a column, so queries filtering on it read only matching directories.
W7	**Bucketing and bucket map join**	Bucketing splits each Hive partition into a fixed number of files by hash(column) % N.
W7	**Presto and the SQL-on-Hadoop engines**	Presto is Meta's in-memory SQL engine that answers interactive queries over HDFS in seconds by never writing intermediate results to disk.
W7	**Case study: Spotify Discover Weekly**	Discover Weekly is Spotify's weekly recommendation playlist, built from playlist co-occurrence in HiveQL and ALS collaborative filtering in Spark, scheduled by Luigi.
W8	**From point-to-point ETL to event pipelines**	Point-to-point ETL connects every pair of systems with its own job, giving $O(N^2)$ jobs.
W8	**Batch ingestion: Sqoop and Flume**	Sqoop is a batch tool that copies tables in bulk between an RDBMS and HDFS.
W8	**Kafka: the append-only log**	A Kafka offset is the integer position of a message inside a partition.
W8	**Why Kafka is fast**	Zero-copy uses sendfile() to send data from disk to the network without copying it through user space.
W8	**Lambda and Kappa architectures**	Kappa architecture drops Lambda's batch layer and reprocesses history by replaying the Kafka log.
W8	**Data lakes and the data swamp**	A data swamp is a data lake with no governance: files without metadata, owners or quality checks.
W9	**Tufte: data-ink, chartjunk, lie factor**	The data-ink ratio is the share of a chart's ink that encodes data, and Tufte says to maximise it.
W9	**Shneiderman's mantra and choosing the chart**	Shneiderman's mantra is "overview first, zoom and filter, then details on demand".

W9	**Overplotting: alpha, hexbin, KDE**	Overplotting is when millions of scatter points overlap into a solid blob that shows nothing.
W9	**Latency: data cubes and AQP**	A data cube is a set of pre-computed aggregate tables over combinations of dimensions, read instead of the raw fact table.
W9	**Streaming dashboards and tools**	Grafana is the tool for real-time ops dashboards that poll a time-series database every few seconds.
W9	**Case study: NYT COVID-19 tracker**	The NYT COVID tracker showed that cleaning data from 50 sources was harder than drawing the charts.
W10	**Social media as a graph: communities, influence, Twitter's shape**	Community detection finds groups of nodes more densely linked to each other than to the rest of the graph, usually with the Louvain algorithm.
W10	**Sentiment (VADER), trend detection and ethics**	VADER is a lexicon-and-rule-based sentiment analyser for social media that outputs a compound score in [−1, +1].
W10	**Time series, ARIMA and Prophet at scale**	Prophet is a time series forecasting model that splits a series into trend, seasonality, holidays and noise: $y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$.
W10	**Anomaly detection: STL and Isolation Forest**	An anomaly is a data point that does not conform to the expected pattern of the series.
W10	**Healthcare big data, privacy and AlphaFold**	Federated learning trains a model at each hospital and shares only model updates, so raw patient records never leave the site.
W10	**Syndromic surveillance and the Google Flu Trends lesson**	Syndromic surveillance uses non-traditional signals, such as posts, pharmacy sales and ER notes, that show illness before a diagnosis is reported.